

Python Sorting Algorithms

Sorting Algorithms

Sorting algorithms are computational methods used to rearrange a list or array of elements in a specific order. They are fundamental for optimizing data organization and searching tasks. Algorithms are categorized into comparison-based methods (e.g., Quicksort, Mergesort) and non-comparison-based methods (e.g., Counting Sort).

Key Types of Sorting Algorithms

Comparison-Based Algorithms: These algorithms rely on comparing elements to each other.

- **Quicksort:** A highly efficient, divide-and-conquer algorithm that partitions an array around a chosen "pivot" element.
- **Mergesort:** A stable, divide-and-conquer approach that recursively breaks down the array, sorts the pieces, and merges them back together.
- **Bubble Sort:** A simple iterative method that repeatedly steps through the list, compares adjacent elements, and swaps them if they are out of order.
- **Insertion Sort:** Builds the final sorted array one element at a time, repeatedly inserting the next element into the correctly sorted portion.

Non-Comparison-Based Algorithms: These use the properties of the data itself without pairwise comparisons.

- **Counting Sort:** Ideal for integers within a specific range, it tallies the occurrences of each number to determine their exact sorted positions.

Algorithm Complexity and Stability

Efficiency is determined by time complexity (how fast it runs based on input size N) and space complexity (how much extra memory is used). Stability refers to whether the algorithm maintains the relative original order of identical elements.

A sorting algorithm is **stable** if it **guarantees that elements with equal values (or keys) maintain their original relative order** from the input list. If an algorithm swaps identical keys arbitrarily, it is considered **unstable**.

Bubble Sort

Bubble Sort is a straightforward comparison-based algorithm that continuously steps through a list, compares adjacent elements, and swaps them if they are in the wrong order. This process repeats until the entire list is completely sorted.

How it works:

- i.) Go through the array, one value at a time.

- ii.) For each value, compare the value with the next value.
- iii.) If the value is higher than the next one, swap the values so that the highest value comes last.
- iv.) Go through the array as many times as there are values in the array.

Manual Run Through

In the example below, we will manually run through a short array (once), to illustrate the sorting process (Bubble Sort).

Step 1: We start with an unsorted array.

[7, 12, 9, 11, 3]

Step 2: We look at the two first values. Does the lowest value come first? Yes, so we don't need to swap them.

[7, 12, 9, 11, 3]

Step 3: Take one step forward and look at values 12 and 9. Does the lowest value come first? No.

[7, 12, 9, 11, 3]

Step 4: So we need to swap them so that 9 comes first.

[7, 9, 12, 11, 3]

Step 5: Taking one step forward, looking at 12 and 11.

[7, 9, 12, 11, 3]

Step 6: We must swap so that 11 comes before 12.

[7, 9, 11, 12, 3]

Step 7: Looking at 12 and 3, do we need to swap them? Yes.

[7, 9, 11, 12, 3]

Step 8: Swapping 12 and 3 so that 3 comes first.

[7, 9, 11, 3, 12]

We then Repeat until no more swaps are needed and you will get a sorted array:

Implementing Bubble Sort in Python

To implement the Bubble Sort algorithm in Python, we need:

- i.) An array with values to sort.
- ii.) An inner loop that goes through the array and swaps values if the first value is higher than the next value. This loop must loop through one less value each time it runs.

iii.) An outer loop that controls how many times the inner loop must run. For an array with n values, this outer loop must run $n-1$ times.

The resulting code looks like this:

Example No Function, No Optimization (BubbleSortOne.py)

```
def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t", mylist)

    n = len(mylist)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if mylist[j] > mylist[j + 1]:
                temp = mylist[j]
                mylist[j] = mylist[j + 1]
                mylist[j + 1] = temp

    print("After Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Stop Press: Iterable unpacking/ multiple assignment

Using more than one variable on each side of the $=$ operator in Python is called iterable unpacking or multiple assignment.

The Core Mechanism

When Python processes an assignment statement, it always evaluates the right-hand side completely first, packs it into a temporary structure (like a tuple), and then unpacks it into the variables on the left-hand side from left to right.

1. Basic Multiple Assignment

You can assign different values to different variables in a single line by separating them with commas.

python

```
x, y, z = 10, 20, 30
```

- **How it works:** Python sees 10, 20, 30 as a single tuple (10, 20, 30). It then unpacks the elements into x, y, and z based on their position.

- **Rule:** The number of variables on the left **must exactly match** the number of elements on the right. Otherwise, Python throws a ValueError.

2. Variable Swapping (No Temporary Variable Needed)

Because the right-hand side is evaluated before any assignment happens, you can swap variables seamlessly.

```
python
```

```
a = 5
```

```
b = 9
```

```
a, b = b, a
```

```
# Now a is 9, b is 5
```

- **How it works:** Python evaluates b, a first, creating a temporary tuple (9, 5). Only then does it assign those values back to a and b. This eliminates the need for a third temp variable.

3. Unpacking Collections

The right-hand side does not have to be a comma-separated list of values; it can be any iterable object like a list, tuple, or string.

```
colors = ["red", "green", "blue"]
```

```
r, g, b = colors
```

- **How it works:** Python takes the items out of the colors list sequentially and matches them to r, g, and b

4. Advanced: Extended Unpacking (Using *)

If you don't know how many elements are on the right-hand side, or if you only want to extract a few specific items, you can use the * (starred) operator to capture the rest.

```
python
```

```
first, *middle, last = [1, 2, 3, 4, 5]
```

```
# first = 1
```

```
# middle = [2, 3, 4]
```

```
# last = 5
```

- **How it works:** first gets the first item, last gets the last item, and *middle scoops up everything left over into a new list.

Example [The above example redone] (BubbleSortTwo.py)

```
def main():
```

```
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]
```

```
print("\nBefore Sorting:\t",mylist)

n = len(mylist)

for i in range(n - 1):
    for j in range(n - i - 1):
        if mylist[j] > mylist[j + 1]:
            mylist[j], mylist[j + 1] = mylist[j + 1], mylist[j]

print("After Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Example (BubbleSortThree.py)

The example below illustrates/traces **all the steps** followed to sort the list using bubble sort.

```
def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t",mylist)

    n = len(mylist)

    print()

    def main():
        mylist = [64, 34, 25, 12, 22, 11, 90, 5]

        print("\nBefore Sorting:\t",mylist)

        n = len(mylist)

        print()

        for i in range(n - 1):
            for j in range(n - i - 1):
                if mylist[j] > mylist[j + 1]:
                    mylist[j], mylist[j + 1] = mylist[j + 1], mylist[j]
```

```

    #The statement below outputs the list after every outer loop iteration
    print(mylist)

    print("-----")

    print("\nAfter Sorting:\t", mylist)

if __name__ == "__main__":
    main()

```

Example (BubbleSortFour.py)

The example below illustrates/traces the stages followed to sort the list at a higher level.

```

def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t",mylist)

    n = len(mylist)

    print()

    for i in range(n - 1):
        for j in range(n - i - 1):
            if mylist[j] > mylist[j + 1]:
                mylist[j], mylist[j + 1] = mylist[j + 1], mylist[j]

    #The statement below outputs the list after every outer loop iteration
    print(mylist)

    print("-----")

    print("\nAfter Sorting:\t", mylist)

if __name__ == "__main__":
    main()

```

Optimizing The Bubble Sort Algorithm

The Bubble Sort algorithm can be improved/optimized a little bit more.

Take an array is almost sorted already, with the lowest numbers at the start, like the one below:

```
mylist = [7, 3, 9, 12, 11]
```

In this case, the array will be sorted after the first run, but the Bubble Sort algorithm will continue to run, without swapping elements, and that is not necessary.

If the algorithm goes through the array one time without swapping any values, the array must be fully sorted, and we can stop the algorithm as shown below:

Example (BubbleSortOptimized.py)

```
def main():
    mylist = [7, 3, 9, 12, 11]

    print("\nBefore Sorting:\t",mylist)

    n = len(mylist)

    print()

    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if mylist[j] > mylist[j + 1]:
                mylist[j], mylist[j + 1] = mylist[j + 1], mylist[j]
                swapped = True
        print(mylist) # Included for illustration purpose only.

    if not swapped:
        break

    print(f"-----")

    print("\nAfter Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Key Logic Steps

- **Outer Loop:** Runs up to n times to manage the total number of sorting passes.
- **Inner Loop:** Performs the actual element comparisons. It shrinks by i elements each pass because the largest unsorted items naturally "bubble up" to the end of the list.
- **Pythonic Swapping:** Instead of relying on a temporary variable, Python uses simultaneous assignment (`mylist[j], mylist[j + 1] = mylist[j + 1], mylist[j]`) to swap values cleanly.

- **Early Stop Optimization:** The swapped flag tracks whether any elements changed places during a pass. If a pass completes with swapped remaining False, the list is verified as sorted and the loops break early

Implementing Bubble Sort Using A User-Defined Function (optimized code)

The example below implements the bubble sort algorithm using a user defined function. The implementation includes an early-stop optimization flag (swapped), which terminates the function immediately if a full pass occurs without any swaps, signaling that the list is already sorted.

Example (BubbleSortWithFnOne.py)

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break

def main():
    sample_list = [64, 34, 25, 12, 22, 11, 90]

    print("\nOriginal list:\t", sample_list)

    bubble_sort(sample_list)

    print("Sorted list:\t", sample_list)

if __name__ == "__main__":
    main()
```

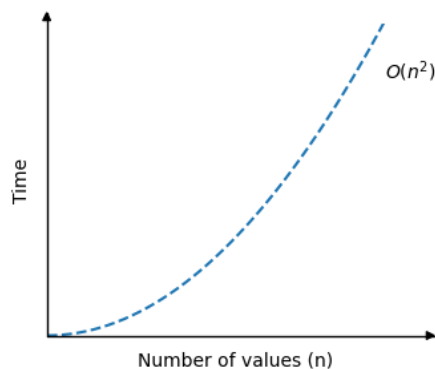
Bubble Sort Time Complexity

The Bubble Sort algorithm loops through every value in the array, comparing it to the value next to it. So for an array of n values, there must be n such comparisons in one loop.

And after one loop, the array is looped through again and again n times (worst case scenario and without optimization).

This means there are $n * n$ comparisons done in total, so the time complexity for Bubble Sort is: $O(n^2)$

The graph describing the Bubble Sort time complexity is shown below:



As clearly indicated, the run time increases really fast when the size of the array is increased. There are sorting algorithms that are faster than this, like Quicksort.

Complexity Analysis

Metric	Time/Space Complexity	Description
Best-case Time	$O(n)$	Occurs when the input list is already sorted, and with the optimization flag.
Average-case Time	$O(n^2)$	Occurs under typical unsorted list conditions.
Worst-case Time	$O(n^2)$	Occurs when the input list is sorted in reverse order.
Space Complexity	$O(1)$ Auxiliary	Bubble sort is an in-place algorithm that modifies the source list directly without extra memory allocation.

Detailed Breakdown

1. Worst-Case Scenario: $O(n^2)$

- **Condition:** Occurs when the input array is completely reversed (e.g., [5, 4, 3, 2, 1]).
- **Explanation:** The algorithm must compare and swap every single adjacent element pair. In the first pass, it makes $(n-1)$ comparisons; in the second, $(n-2)$ comparisons, and so on.
- **Total Operations:** The total number of comparisons equals the sum of the first $(n-1)$ integers:

$$\text{Total Comparisons} = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

In Big O notation, lower-order terms and constants are dropped, leaving $O(n^2)$.

2. Average-Case Scenario: $O(n^2)$

- **Condition:** Occurs when elements are randomly distributed.

- **Explanation:** Even if some parts of the array are partially sorted, the algorithm still requires nested loops to iterate through the remaining unsorted elements. The total number of operations remains proportional to n^2 .

3. Best-Case Scenario: $O(n)$ (Optimized)

- **Condition:** Occurs when the input array is already completely sorted (e.g., [1, 2, 3, 4, 5]).
- **Explanation:** In a **standard (naive)** bubble sort, the algorithm will run through all nested loops regardless, resulting in $O(n^2)$. However, in an **optimized** bubble sort, a tracking boolean flag is added to check if a swap happened during a pass. If the first pass completes with **zero swaps**, the algorithm recognizes the array is sorted and terminates early after exactly $(n-1)$ comparisons.

Stop Press: Time Complexity

Time complexity

Time complexity is a theoretical measure that describes the amount of computer time an algorithm takes to run as a function of the length of the input. It counts the number of elementary operations executed by the code rather than measuring actual clock time in seconds, ensuring the evaluation remains independent of specific hardware, compilers, or programming languages.

Common Asymptotic Notations

- Big O (O)**: Represents the **worst-case scenario** or the absolute upper bound of execution time.
- Big Omega (Ω)**: Represents the **best-case scenario** or the absolute lower bound of execution time.
- Big Theta (Θ)**: Represents the **average-case scenario** or the tight bound when upper and lower bounds match.

How to calculate the time complexity of the bubble sort algorithm

The **worst-case time complexity of Bubble Sort is $O(n^2)$** where n represents the total number of elements in the array.

Below is the exact step-by-step mathematical breakdown to calculate this complexity by counting the total number of element comparisons.

1. Count Comparisons Per Pass

Bubble sort operates in a sequence of passes. In each pass, the algorithm compares adjacent elements and swaps them if they are out of order.

- **Pass 1:** Compares all elements to push the largest item to the end. This takes $n - 1$ comparisons.

- **Pass 2:** The largest element is already in place, so it checks the remaining $n - 1$ items. This takes $n - 2$ comparisons.
- **Pass 3:** The two largest elements are sorted. This takes $n - 3$ comparisons.
- ...
- ...
- **Final Pass:** Only the last two remaining unsorted elements are compared. This takes 1 comparison.

2. Set Up the Summation

To find the grand total of comparisons made across the entire algorithm, sum the number of comparisons from every single pass:

$$\text{Total Comparisons} = (n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

3. Evaluate the Arithmetic Series

The expression above is a standard arithmetic series. Use the mathematical summation formula for the first $n - 1$ integers, which is defined as $\frac{k(k+1)}{2}$ where $k = n - 1$:

$$\text{Total} = \frac{(n - 1)((n - 1) + 1)}{2}$$

$$\text{Total} = \frac{n(n - 1)}{2}$$

$$\text{Total} = \frac{n^2 - n}{2}$$

4. Apply Big O Notation

To express this result using asymptotic Big O notation, isolate the fastest-growing term as n approaches infinity and discard all coefficients and lower-order terms:

- Drop the lower-order term $-n$.
- Drop the constant multiplier fractional coefficient $(\frac{1}{2})$

This isolates the dominant term, which leaves: $O(n^2)$

Selection Sort

The Selection Sort algorithm finds the lowest value in an array and moves it to the front of the array.

The algorithm looks through the array again and again, moving the next lowest values to the front, until the array is sorted.

The Selection Sort algorithm divides an input list into two sections: a sorted portion at the front and an unsorted portion at the back. The algorithm repeatedly scans the unsorted section to find the smallest element and swaps it into its correct position at the end of the sorted section (front of the unsorted section).

How it works:

- i.) Go through the array to find the lowest value.
- ii.) Move the lowest value to the front of the unsorted part of the array.
- iii.) Go through the array again as many times as there are values in the array.

Manual Run Through

In the example below, we will manually run through a short array, to illustrate the sorting process (Selection Sort).

Sorting an unsorted array of five numbers: **[29, 10, 14, 37, 13]**

Pass 1

- **Current state:** [29, 10, 14, 37, 13] (Entire array is unsorted).
- **Action:** Search the whole array for the smallest number.
- **Scan:**
 - ❖ We start with 29.
 - ❖ Compare with 10. 10 is smaller; We swap.
[10, 29, 14, 37, 13]
 - ❖ Compare 10 with 14. Not smaller.
 - ❖ Compare 10 with 37. Not smaller.
 - ❖ Compare 10 with 13. Not smaller than 10.
- **Smallest found:** 10
- **Result:** [10 | 29, 14, 37, 13] (The 10 is now in the sorted region).

Pass 2

- **Current state:** [10 | 29, 14, 37, 13]
- **Action:** Find the smallest number in the remaining unsorted portion (29, 14, 37, 13).
- **Scan:**
 - ❖ We start with 29.
 - ❖ Compare 29 with 14. 14 is smaller; we swap.

[10 | 14, 29, 37, 13]

- ❖ Compare 14 with 37. Not smaller.
- ❖ Compare 14 with 13. 13 is smaller; We swap

[10 | 13, 29, 37, 14]

- **Smallest found:** 13
- **Result:** [10, 13 | 14, 37, 29]

Pass 3

- **Current state:** [10, 13 | 14, 37, 29]
- **Action:** Find the smallest number in the unsorted portion (14, 37, 29).
- **Scan:**
 - ❖ We start with 14.
 - ❖ Compare 14 with 37. Not smaller.
 - ❖ Compare 14 with 29. Not smaller.
- **Smallest found:** 14
- 14 is already in its correct position.
- **Result:** [10, 13, 14 | 37, 29]

Pass 4

- **Current state:** [10, 13, 14 | 37, 29]
 - **Action:** Find the smallest number in the final two unsorted elements (37, 29).
 - **Scan:**
 - ❖ We start with 37.
 - ❖ Compare 37 with 29. 29 is smaller. We swap.
- [10, 13, 14 | 29, 37]

- **Smallest found:** 29
- **Result:** [10, 13, 14, 29 | 37]

Final Array

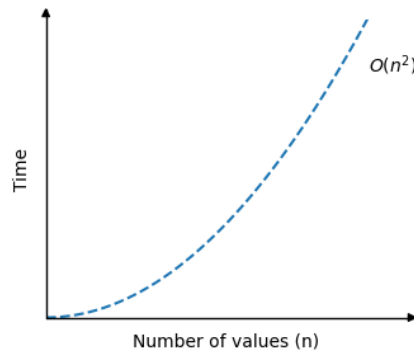
Since only one element (37) remains in the unsorted region, it must be the largest and is automatically in its correct position.

The fully sorted array is: [10, 13, 14, 29, 37]

Key Technical Properties

- **Time Complexity:** Always $O(n^2)$ for best, worst, and average cases. The nested loops check all items regardless of the array's initial order.
- **Space Complexity:** $O(1)$ auxiliary space. It sorts the elements entirely in place without utilizing temporary arrays.
- **Swapping efficiency:** Selection sort performs a maximum of $n - 1$ swaps, making it useful when writing to memory is highly expensive

The time complexity for the Selection Sort algorithm can be displayed in a graph as shown below:



The run time for selection sort is the same as for Bubble Sort: The run time increases really fast when the size of the array is increased.

Implementing Selection Sort in Python

To implement the Selection Sort algorithm in Python, we need:

- An array with values to sort.
- An inner loop that goes through the array, finds the lowest value, and moves it to the front of the array. This loop must loop through one less value each time it runs.
- An outer loop that controls how many times the inner loop must run. For an array with n values, this outer loop must run $n - 1$ times.

The resulting code looks like this:

Example (SelectionSortOne.py)

```
def main():
    numbers = [64, 25, 12, 22, 11]

    n = len(numbers)

    print("\nBefore Sorting:\t", numbers)

    # Traverse through all array elements
    for i in range(n - 1):
```

```
# Assume the current position i holds the minimum element
min_index = i

# Test against the remaining unsorted elements
for j in range(i + 1, n):
    if numbers[j] < numbers[min_index]:
        min_index = j

# Swap the found minimum element with the first unsorted element
if min_index != i:
    numbers[i], numbers[min_index] = numbers[min_index], numbers[i]

print("\nAfter Sorting:\t",numbers)

if __name__ == "__main__":
    main()
```

Example (SelectionSortTwo.py)

```
def main():
    numbers = [64, 25, 12, 22, 11]

    n = len(numbers)

    print("\nBefore Sorting:\t",numbers)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if numbers[j] < numbers[min_index]:
                min_index = j

        if min_index != i:
            temp = numbers[i]
            numbers[i] = numbers[min_index]
            numbers[min_index] = temp

    print("\nBefore Sorting:\t",numbers)

if __name__ == "__main__":
```

```
main()
```

Example (SelectionSortThree.py)

The example below illustrates/traces **all the steps** followed to sort the list using selection sort.

```
def main():
    numbers = [64, 25, 12, 22, 11]

    n = len(numbers)

    print("\nBefore Sorting:\t",numbers)

    print()

    for i in range(n - 1):
        for j in range(i + 1, n):
            if numbers[j] < numbers[i]:
                numbers[j], numbers[i] = numbers[i], numbers[j]

        print(numbers)
        print("-----")

    print("\nAfter Sorting:\t",numbers)

if __name__ == "__main__":
    main()
```

Implementing Selection Sort Using a User-Defined Function

The example below implements the selection sort algorithm using a user defined function.

Example (SelectionSortWithFnOne.py)

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
```



```
arr[i], arr[min_index] = arr[min_index], arr[i]

def main():
    numbers = [64, 25, 12, 22, 11]

    print("\nBefore Sorting:\t", numbers)

    selection_sort(numbers)

    print("\nAfter Sorting:\t", numbers)

if __name__ == "__main__":
    main()
```

Example (SelectionSortWithFnTwo.py)

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(i + 1, n):
            if arr[j] < arr[i]:
                arr[j], arr[i] = arr[i], arr[j]

def main():
    numbers = [64, 96, 25, 12, 32, 9, 22, 11]

    print("\nBefore Sorting:\t", numbers)

    selection_sort(numbers)

    print("\nAfter Sorting:\t", numbers)

if __name__ == "__main__":
    main()
```

Insertion Sort

The Insertion Sort algorithm uses one part of the array to hold the sorted values, and the other part of the array to hold values that are not sorted yet.

The algorithm takes one value at a time from the unsorted part of the array and puts it into the right place in the sorted part of the array, until the array is sorted.

insertion sort in Python can be implemented by iterating through an array from the second element to the last, shifting larger elements to the right to find the correct placement for each item.

How it works

- i.) Take the first value from the unsorted part of the array.
- ii.) Move the value into the correct place in the sorted part of the array.
- iii.) Go through the unsorted part of the array again as many times as there are values.

Manual Run Through

In the example below, we will manually run through a short array, to illustrate the sorting process (Insertion Sort).

Step-by-step walk-through using the unsorted array: [5, 3, 8, 4, 2].

1. Initialize the Sorted Section

- **Action:** Start at the very first element (index 0).
- **State:** [5 | 3, 8, 4, 2]
- **Detail:** A single element [5] is always sorted by default. The vertical bar | separates the sorted portion (left) from the unsorted portion (right).

2. Insert the Second Element

- **Target Value:** 3 (index 1)
- **Comparison:** Compare 3 with the sorted element on its left, 5.
- **Shift:** Because $5 > 3$, shift 5 to the right to clear space.
- **Insertion:** Insert 3 into the first position.
- **Result:** [3, 5 | 8, 4, 2]

3. Insert the Third Element

- **Target Value:** 8 (index 2)
- **Comparison:** Compare 8 with the sorted element on its left, 5.
- **Shift:** Because $5 < 8$, no shifting is needed. The element is already in the correct relative position.
- **Insertion:** Keep 8 exactly where it is.
- **Result:** [3, 5, 8 | 4, 2]

4. Insert the Fourth Element

- **Target Value:** 4 (index 3)

- **Comparison 1:** Compare 4 with 8. Since $8 > 4$, shift 8 to the right.
- **Comparison 2:** Compare 4 with 5. Since $5 > 4$, shift 5 to the right.
- **Comparison 3:** Compare 4 with 3. Since $3 < 4$, stop shifting.
- **Insertion:** Insert 4 into the vacant slot right after 3.
- **Result:** [3, 4, 5, 8 | 2]

5. Insert the Fifth Element

- **Target Value:** 2 (index 4)
- **Shifting Process:** Compare 2 sequentially against 8, 5, 4, and 3 (As illustrated in step 4 above). Because 2 is smaller than all of them, shift every single one of these elements (one by one) one position to the right.
- **Insertion:** Place 2 at the very beginning of the array (index 0).
- **Final Result:** [2, 3, 4, 5, 8]

Summary of Performance

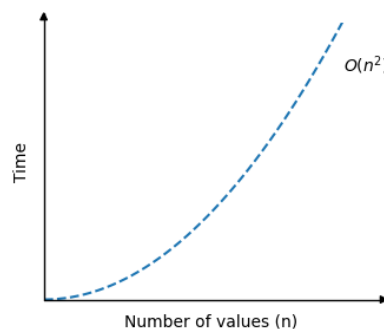
- **Best-Case Time Complexity:** $O(n)$ when the array is already sorted, because it only makes comparisons without shifts.
- **Worst-Case Time Complexity:** $O(n^2)$ when the array is in reverse order, forcing every element to shift completely to the left.
- **Space Complexity:** $O(1)$ auxiliary space because it sorts the items in-place without needing extra arrays.

Insertion Sort sorts an array of n values.

On average, each value must be compared to about $\frac{n}{2}$ other values to find the correct place to insert it.

Insertion Sort must run the loop to insert a value in its correct place approximately n times.

We get time complexity for Insertion Sort: $O\left(\frac{n}{2} \cdot n\right) = O(n^2)$



For Insertion Sort, there is a big difference between best, average and worst case scenarios.

Implementing Insertion Sort in Python

To implement the Insertion Sort algorithm in a Python program, we need:

- i.) An array with values to sort.
- ii.) An outer loop that picks a value to be sorted. For an array with n values, this outer loop skips the first value, and must run $n - 1$ times.
- iii.) An inner loop that goes through the sorted part of the array, to find where to insert the value. If the value to be sorted is at index i the sorted part of the array starts at index 0 and ends at index $i - 1$

Below is the resulting code.

Example (InsertionSortOne.py)

```
def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t",mylist)

    n = len(mylist)

    for i in range(1, n):
        insert_index = i
        current_value = mylist[i]

        for j in range(i - 1, -1, -1):
            if mylist[j] > current_value:
                mylist[j + 1] = mylist[j]
                insert_index = j
            else:
                break

        mylist[insert_index] = current_value

    print("After Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Example (InsertionSortTwo.py)

The example below illustrates/traces **all the steps** followed to sort the list using insertion sort.

```
def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t",mylist)

    n = len(mylist)
    print("\n")

    for i in range(1, n):
        insert_index = i
        current_value = mylist[i]

        for j in range(i - 1, -1, -1):
            if mylist[j] > current_value:
                mylist[j + 1] = mylist[j]
                mylist[j] = "_" #This line added for illustration only
                insert_index = j
                print(mylist)
            else:
                break

        mylist[insert_index] = current_value
        print(mylist)
        print("-----")

    print("\n\nAfter Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Example (InsertionSortThree.py)

The example below uses the pop() and insert() methods to implement the insertion sort. This approach is not optimal because it involves very many shift operations in the background.

```
def main():
    mylist = [64, 34, 25, 12, 22, 11, 90, 5]

    print("\nBefore Sorting:\t",mylist)
```

```

n = len(mylist)

for i in range(1,n):
    insert_index = i
    current_value = mylist.pop(i)

    for j in range(i - 1, -1, -1):
        if mylist[j] > current_value:
            insert_index = j

    mylist.insert(insert_index, current_value)

print("After Sorting:\t", mylist)

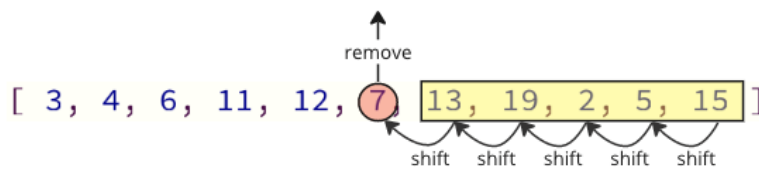
if __name__ == "__main__":
    main()

```

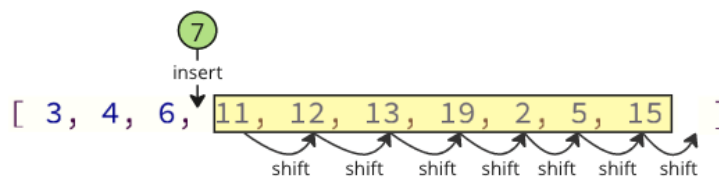
Background explanation of the code above

The way the code above first removes a value and then inserts it somewhere else is intuitive. It is how you would do Insertion Sort physically with a hand of cards for example. If low value cards are sorted to the left, you pick up a new unsorted card, and insert it in the correct place between the other already sorted cards.

The problem with this way of programming it is that when removing a value from the array, all elements above must be shifted one index place down:



And when inserting the removed value into the array again, there are also many shift operations that must be done: all following elements must shift one position up to make place for the inserted value:

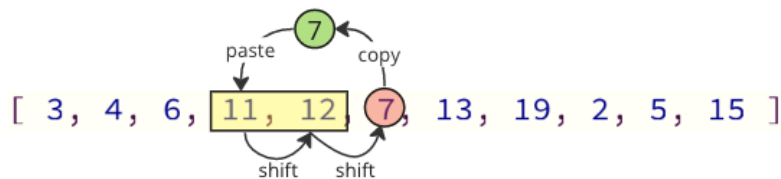


These shifting operations can take a lot of time, especially for an array with many elements.

Hidden memory shifts: You will not see these shifting operations happening in the code if you are using a high-level programming language such as Python or JavaScript, but the shifting operations are still happening in the background. Such shifting operations require extra time for the computer to do, which can be a problem.

Review of the first example (InsertionSortOne.py)

In the earlier example (InsertionSortOne.py) most of these shift operations are avoided by only shifting the values necessary.



In the image above, first value 7 is copied, then values 11 and 12 are shifted one place up in the array, and at last value 7 is put where value 11 was before.

The number of shifting operations is reduced from 12 to 2 in this case.

This is a much more efficient approach.

Example (InsertionSortFour.py)

The example below uses the while loop instead of for loop in the inner loop

```
def main():
```

```
    numbers = [56, 134, 33, 265, 912, 242, 4, 29, 189, 16]
```

```
    print("\nBefore Sorting:\t", numbers)
```

```
    n = len(numbers)
```

```
    for i in range(1, n):
```

```
        current_value = numbers[i] # The element to be positioned
```

```
        j = i - 1
```

```
        # Move elements of numbers[0..i-1] that are greater than current_value
```

```
        # to one position ahead of their current position
```

```
        while j >= 0 and numbers[j] > current_value:
```

```
            numbers[j + 1] = numbers[j]
```

```
            j = j - 1
```

```
        # Insert the current_value into its correct sorted location
```

```
    numbers[j + 1] = current_value

print("After Sorting:\t", numbers)

if __name__ == "__main__":
    main()
```

Example (InsertionSortFive.py)

The example below modifies the while loop and introduces the if statement.

```
def main():
    numbers = [56, 134, 33, 265, 912, 242, 4, 29, 189, 16]

    print("\nBefore Sorting:\t", numbers)

    n = len(numbers)

    for i in range(1, n):
        current_value = numbers[i] # The element to be positioned
        j = i - 1

        # Move elements of numbers[0..i-1] that are greater than current_value
        # to one position ahead of their current position

        while j >= 0:
            if numbers[j] > current_value:
                numbers[j + 1] = numbers[j]
            else:
                break;
            j = j - 1

        # Insert the current_value into its correct sorted location
        numbers[j + 1] = current_value

    print("After Sorting:\t", numbers)

if __name__ == "__main__":
    main()
```

Implementing Insertion Sort Using a User-Defined Function

The example below implements the insertion sort algorithm using a user defined function.

Example (InsertionSortWithFnOne.py)

```
def insertion_sort(numbers):
    n = len(numbers)

    for i in range(1, n):
        insert_index = i
        current_value = numbers[i]

        for j in range(i - 1, -1, -1):
            if numbers[j] > current_value:
                numbers[j + 1] = numbers[j]
                insert_index = j
            else:
                break

        numbers[insert_index] = current_value

def main():
    mylist = [58, 36, 29, 19, 4, 14, 21, 11, 89, 9]

    print("\nBefore Sorting:\t", mylist)

    insertion_sort(mylist)

    print("After Sorting:\t", mylist)

if __name__ == "__main__":
    main()
```

Merge Sort

Merge Sort is a highly efficient, stable, and comparison-based sorting algorithm that operates on the **Divide and Conquer** paradigm. It repeatedly divides an unsorted list into halves until individual elements are isolated, then merges those sub-lists back together in a sorted structure.

Divide: The algorithm starts with breaking up the array into smaller and smaller pieces until one such sub-array only consists of one element.

Conquer: The algorithm merges the small pieces of the array back together by putting the lowest values first, resulting in a sorted array.

The breaking down and building up of the array to sort the array is done recursively.

How it works:

- i.) Divide the unsorted array into two sub-arrays, half the size of the original.
- ii.) Continue to divide the sub-arrays as long as the current piece of the array has more than one element.
- iii.) Merge two sub-arrays together by always putting the lowest value first.
- iv.) Keep merging until there are no sub-arrays left.

The diagram below illustrates how Merge Sort works. The array is split into smaller and smaller pieces until it is merged back together. And as the merging happens, values from each sub-array are compared so that the lowest value comes first.

